```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
```

```
df=pd.read_csv("/content/breast_cancer_data.csv")
print("Dataset shape:",df.shape)
df.head()
```

```
Dataset shape: (569, 33)
```

	id	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean	compactness_mean	concavity_m
0	842302	M	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.3
1	842517	M	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.0
2	84300903	M	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.1
3	84348301	M	11.42	20.38	77.58	386.1	0.14250	0.28390	0.2
4	84358402	M	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.1

5 rows × 33 columns

```
df = df.drop(columns=["id", "Unnamed: 32"], errors="ignore")
df["diagnosis"] = df["diagnosis"].map({"M":1, "B":0})

X = df.drop(columns=["diagnosis"])
y = df["diagnosis"]

print("Features shape:", X.shape)
print("Target distribution:\n", y.value_counts())
```

```
Features shape: (569, 30)
Target distribution:
 diagnosis
0    357
1    212
Name: count, dtype: int64
```

```
X_train, X_test, y_train, y_test = train_test_split(
X, y, test_size=0.2, random_state=42, stratify=y

)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

print("Training set:", X_train.shape)
print("Testing set:", X_test.shape)
```

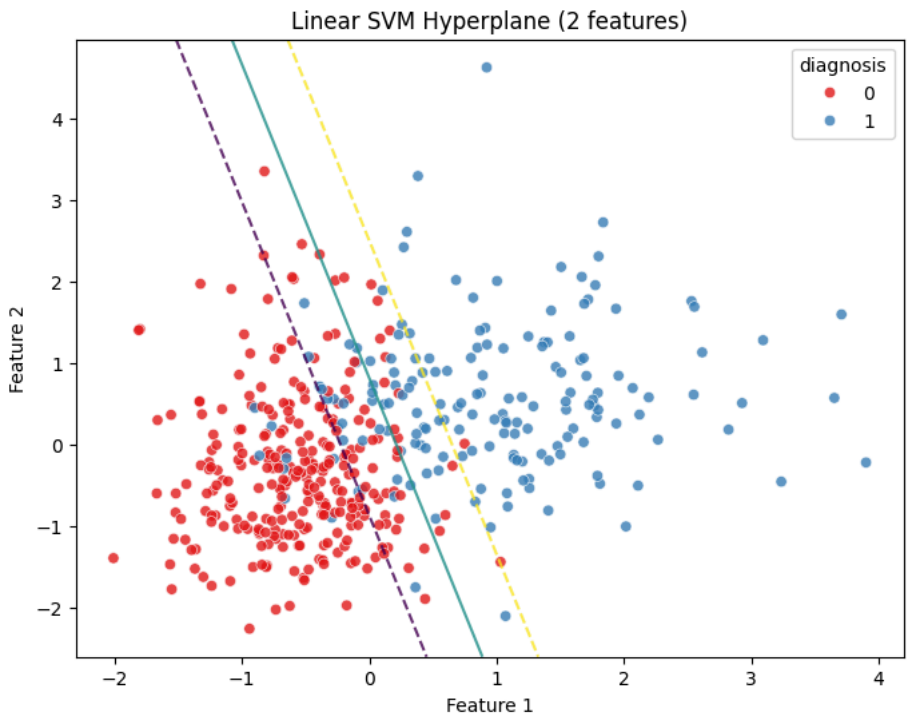
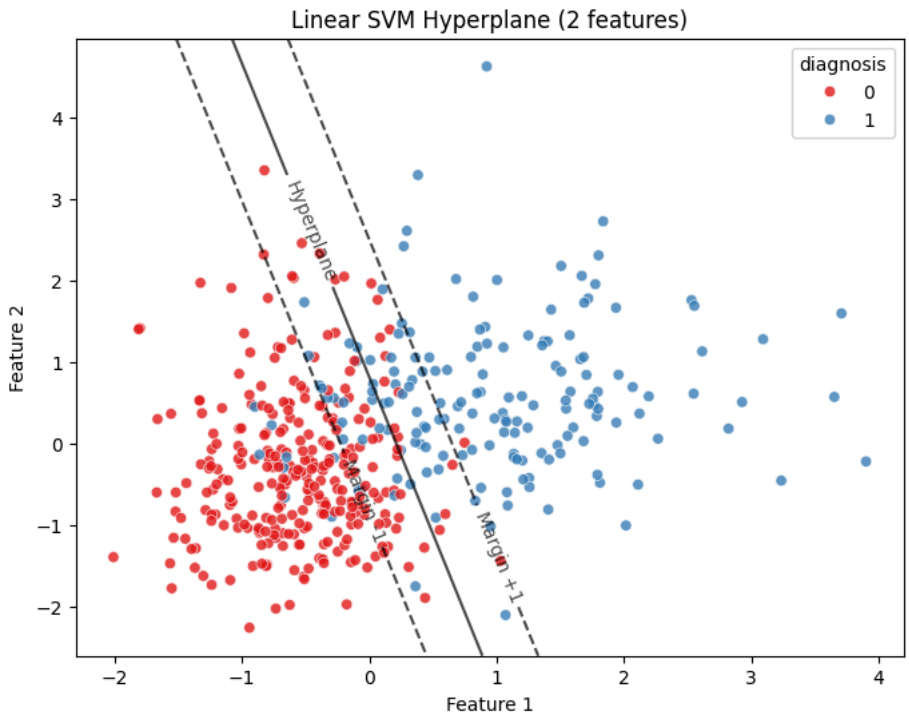
```
Training set: (455, 30)
Testing set: (114, 30)
```

```
kernels = ["linear", "poly", "rbf" ]
results = {}

for k in kernels:
    print(f"\n ---- Training with {k} kernel ---- ")
model = SVC(kernel=k, C=1.0, gamma="scale", degree=3)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

acc = accuracy_score(y_test, y_pred)
results[k] = acc

print("Accuracy:", acc)
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred) )
print("Classification Report:\n", classification_report(y_test, y_pred) )
```

```
 ---- Training with linear kernel ----
```

```
 ---- Training with poly kernel ----

 ---- Training with rbf kernel ----
Accuracy: 0.9736842105263158
Confusion Matrix:
 [[72  0]
 [ 3 39]]
Classification Report:
              precision    recall  f1-score   support

           0       0.96      1.00      0.98        72
           1       1.00      0.93      0.96        42

    accuracy                           0.97       114
   macro avg       0.98      0.96      0.97       114
weighted avg       0.97      0.97      0.97       114
```

```
X_vis = X_train[:, :2]
y_vis = y_train

model_lin = SVC(kernel="linear", C=1.0)
model_lin.fit(X_vis, y_vis)

plt.figure(figsize=(8, 6))
sns.scatterplot(x=X_vis[:, 0], y=X_vis[:, 1], hue=y_vis, palette="Set1", alpha=0.8)

ax = plt.gca()
xlim = ax.get_xlim()
ylim = ax.get_ylim()

xx, yy = np.meshgrid(
    np.linspace(xlim[0], xlim[1], 100),
    np.linspace(ylim[0], ylim[1], 100)
)

Z = model_lin.decision_function(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape)

plt.contour(
    xx, yy, Z,
    levels=[-1, 0, 1],
    alpha=0.8,
    linestyles=['--', '-', '--']
)

plt.title("Linear SVM Hyperplane (2 features)")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.show()
```



```
model_lin = SVC(kernel="linear", C=1.0)
model_lin.fit(X_vis, y_vis)

plt.figure(figsize=(8,6))
sns.scatterplot(x=X_vis[:,0], y=X_vis[:,1], hue=y_vis, palette="Set1", alpha=0.8)

ax = plt.gca()
xlim = ax.get_xlim()
ylim = ax.get_ylim()

xx, yy = np.meshgrid(
    np.linspace(xlim[0], xlim[1], 100),
    np.linspace(ylim[0], ylim[1], 100)
)

Z = model_lin.decision_function(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape)


contours = ax.contour(
    xx, yy, Z,
    colors='k',
    levels=[-1, 0, 1],
    alpha=0.7,
    linestyles=['--', '-', '--']
)


ax.clabel(
    contours,
    inline=True,
    fontsize=10,
    fmt={-1: "Margin -1", 0: "Hyperplane", 1: "Margin +1"}
)

plt.title("Linear SVM Hyperplane (2 features)")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.show()
```